Jyotish LoRA: Implementation Plan

Version	1.2
Date	2026-10-05
Governed by	Project Constitution
Implements	Technical Specification

Progress (2026-10-06)

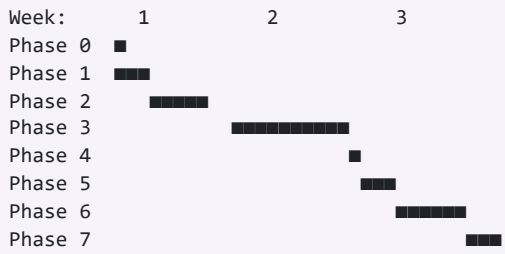
Phase	Status	Evidence
0 Setup	Done, except the Anthropic API key and Kaggle account	Python 3.11 venv in the user profile (`.venvs/astro` , outside OneDrive); Tesseract 5.4 (English only)
1 Corpus	**Done; Gate 1 passed**	99.0% usable pages; 6 books excluded (Lal Kitab x4 and Chamatkar Chintamani are Hindi; Parashara's Light manual is software documentation); 4,012 good chunks; a 30-chunk manual check found no headers or footers; every core topic has 5+ chunks. See `runs/extraction_report.md` and `runs/chunking_report.md`
2 Chart engine	**Done; Gate 2 passed with one open item**	79 tests: well-known sidereal ingresses, Lahiri ayanamsa, lagna at sunrise, nakshatra, navamsa, dignity, dasha, and a positive and negative test for all 33 yogas. **Open:** compare 10 charts against Jagannatha Hora by hand
3 Dataset	**Round 1 done; model review done; human spot-check pending**	879 seeds -> 2,116 examples (questions: Claude Sonnet 5.5; answers: Claude Opus 5.5; fact checks: Gemini 3.1 Pro + Claude Sonnet 5.5). Automatic filters kept 1,885 (89%) after re-checking with refined fact-check rules; a full model review (Sonnet 5.5) approved 1,547 (82%); rejects kept on disk, out of training. Split: 1,420 train / 67 val / 60 test, 0 leaks. **Open:** your 30-example spot-check (`data/reviews/spot_check.csv`)
4 Smoke test	**Ready to run**	`notebooks/train_lora.ipynb` (SMOKE_TEST switch) and the upload folder `kaggle/astro-sft/` are prepared

Decision for task 1.3: Lal Kitab is Hindi (scanned, or text in a legacy font encoding), so it is excluded from v1. Hindi OCR is a candidate for v2.

1. Summary

The plan has **7 phases** over about **5–6 weeks** of part-time work. GPU work happens only on the **Kaggle free tier** (about 30 GPU hours per week). Everything else runs on the laptop. Each phase ends with a **gate**: a checklist that must pass before the next phase starts.

Phase	Name	Where	Duration	GPU hours
0	Setup	Laptop	1 day	0
1	Corpus extraction and chunking	Laptop	3–4 days	0
2	Chart engine	Laptop	4–5 days	0
3	Dataset generation and review	Laptop + teacher LLM	8–12 days	0
4	Smoke test (Qwen3-1.7B)	Kaggle	1 day	about 1
5	Main LoRA training (Qwen3-4B)	Kaggle	2–4 days	about 5–10
6	Evaluation and iteration	Kaggle + laptop	5–7 days	about 5–15
7	Export and app	Laptop	2–3 days	0



Phase 0: Setup

Goal: a reproducible project skeleton.

#	Task	Output
0.1	Run `git init` in `Astro/` and add `.gitignore` (`data/`, `*.gguf`, `JioAICloud-Download/`)	Repository
0.2	Create the folder layout from Tech Spec Section 6	Folders
0.3	Create a Python 3.11 virtual environment; install `anthropic pymupdf pytesseract pyswisseph datasets transformers tqdm datasketch langdetect`	`requirements.txt`
0.4	Install Tesseract 5 with the `eng` and `hin` language packs	`tesseract --version` works
0.5	Create a Kaggle account, verify the phone number (needed for GPU access), install the Kaggle CLI and API token	`kaggle datasets list` works
0.6	Confirm that RAG is out of scope (Constitution v1.1); the corpus is used only offline to generate data	Decision recorded
0.7	Teacher LLM **decided: Claude Opus 5.5** (`claude-opus-5-5`). Still to do: set up the Anthropic API key, `pip install anthropic`, and confirm Anthropic's terms allow this use of outputs (Constitution VI.4)	API works; terms check recorded

Gate 0: the virtual environment is set up, Tesseract works, Kaggle works, and the teacher LLM is chosen.

Phase 1: Corpus extraction and chunking

Goal: clean, chunked text from all 52 PDFs.

#	Task	Output
1.1	Implement `extract.py` (PyMuPDF; OCR fallback for pages under 30 words)	`data/raw_text/*.jsonl`
1.2	OCR the 4 scanned PDFs (`ashtaka-varga` , `astrology-and-stock-market-forecasting` , `greatness-gayatri-jyotish` , `jyotisha-siddhanta-sara`)	OCR text
1.3	Check which script `lal-kitab-*` uses; configure `hin` / `urd` OCR or exclude it from v1	Decision recorded
1.4	Write the extraction report: pages, OCR pages, empty pages, and a 3-page sample for each book	`runs/extraction_report.md`
1.5	Implement `chunk.py` (cleaning, chapter detection, 500–1,000-token chunks)	`data/chunks/chunks.jsonl`
1.6	Implement `topics.py` : tag each chunk with topics (houses, planets, signs, nakshatras, yogas, dasha, remedies)	`topics` field
1.7	Read 30 random chunks by hand to check quality	Notes

Gate 1: - ☐ 90% or more of pages have usable text. - ☐ Books with more than 20% unreadable pages are excluded or fixed. - ☐ No headers or footers remain in the 30 sampled chunks. - ☐ Every core topic (12 houses, 9 planets, 12 signs, major yogas, Vimshottari) has 5 or more chunks.

Phase 2: Chart engine

Goal: deterministic chart facts (Constitution II).

#	Task	Output
2.1	`engine.py` : sidereal (Lahiri) positions for the 9 grahas, lagna, whole-sign houses, nakshatra and pada	Chart JSON
2.2	Dignity, retrograde and combustion flags; graha drishti (Parashari aspects)	Fields in JSON
2.3	Vimshottari dasha (maha and antar) with dates	`dasha` field
2.4	D9 Navamsa (D10 optional)	`vargas` field
2.5	`yogas.py` : 30 or more rule-based yogas	`yogas` field
2.6	`render.py` : `render_chart_text()` (shared by training and serving)	Text block
2.7	Synthetic birth generator with `cities.csv` (about 200 cities, weighted towards India) and a fixed seed	`gen_births(n, seed)`
2.8	Unit tests: 10 well-known charts checked against reliable software (for example Jagannatha Hora), with positions within ±0.1° and correct dashas	`tests/test_chart_engine.py`

Gate 2: - ☐ All 10 reference charts match. - ☐ Every yoga has at least one positive and one negative test. - ☐ `render_chart_text` output is reviewed and readable.

Phase 3: Dataset generation and review

Goal: about 6,000 high-quality instruction examples (Tech Spec Section 5.4). The teacher reads the source passage; the training example does not include it.

#	Task	Output
3.1	Write `prompts/system_v1.txt`: persona, attribution by tradition or text (no chapter or verse numbers), answer structure, safety rules	System prompt
3.2	Write teacher prompts for each task type (7 types)	`prompts/teacher/*.txt`
3.3	Write 50 safety test prompts **by hand** and build a 100-question rule-fidelity quiz from the reference tables (eval only, never used in training)	`evalsets/safety_eval.jsonl`, `evalsets/quiz.jsonl`
3.4	Implement `generate.py`: coverage sampling → 2–3 questions per chunk → answer drafted from the hidden source passage	Raw examples
3.5	**Pilot:** generate 100 examples and review all of them; tune the prompts	Pilot report
3.6	Generate the full set (about 7,500 raw, so about 6,000 remain after filtering), in batches of 500	`data/sft/raw/*.jsonl`
3.7	Implement `filters.py`: fact check against the source passage, citation ban, chart consistency, length, language, MinHash deduplication	Filtered set
3.8	Human review: export a sample of 300 or more to a spreadsheet; approve, edit or reject	`data/reviews/*.csv`
3.9	`split.py`: split 90/5/5, grouped by chunk_id and chart seed	`train.jsonl`, `val.jsonl`, `test.jsonl`
3.10	Upload `train` and `val` as the private Kaggle Dataset `astro-sft` (version 1)	Kaggle dataset

Gate 3: - ☐ 5,000 or more examples after filtering. - ☐ Every core topic is covered by 10 or more examples. - ☐ Task mix within ±3% of the targets. - ☐ Human-review rejection rate under 10%. - ☐ No overlap between the splits (checked automatically). - ☐ No real personal birth data. - ☐ The dataset hash is recorded.

Phase 4: Smoke test on Kaggle (Qwen3-1.7B)

Goal: prove the training pipeline works before spending real GPU quota.

#	Task	Output
4.1	Create `notebooks/train_lora.ipynb` (PEFT + TRL, settings from Tech Spec Section 5.5) and pin library versions	Notebook
4.2	Run with `Qwen/Qwen3-1.7B`, 200 examples, `max_steps=30`	Loss goes down
4.3	Check: completion-only masking is active, GPU memory is logged, a checkpoint saves, and **resume from checkpoint** works	Checklist
4.4	Generate 5 sample answers to confirm the chat template and output format	Samples

Gate 4: the run finishes, loss decreases, resume works, and the outputs are in the correct format.

Phase 5: Main LoRA training (Qwen3-4B-Instruct-2507)

Goal: the first real adapter, `astro-lora-v1`.

#	Task	Output
5.1	Switch to `Qwen/Qwen3-4B-Instruct-2507`, r=32, alpha=64, 3 epochs, LR 2e-4, `max_length` 3072, batch 1 x 8 accumulation	Configuration
5.2	Run for 50 steps interactively to check GPU memory (target under 13.5 GB)	Memory log
5.3	Start a full background run (**Save & Run All**)	Training run
5.4	Watch train and validation loss; if validation loss rises for 2 evals in a row, use the earlier checkpoint	Best checkpoint
5.5	Save the adapter and `run_manifest.json`; download them to `runs/v1/`	`astro-lora-v1`

Gate 5: final validation loss is below the base model's validation loss, the manifest is complete, and the adapter is stored safely.

Phase 6: Evaluation and iteration

Goal: meet the release thresholds (Tech Spec Section 10).

#	Task	Output
6.1	Generate answers on the test set, quiz and safety set for the **base model** and the **adapter**, with the same system prompt	Prediction files
6.2	Automatic metrics: citation-ban rate, chart consistency, quiz accuracy	Scores
6.3	LLM-judge rubric scoring and a pairwise win rate on 100 items	Scores
6.4	A person reviews 30 regression prompts	Notes
6.5	Write `runs/v1/eval_report.md`	Report
6.6	**If below threshold:** find the cause (data or hyperparameters), change **one** thing at a time, retrain as v2, and compare	v2, v3…

Iteration options, in order of priority:

1. Fix data problems found in the failures (for example add more wordings for rules the model gets wrong, fix the teacher prompt, tighten the filters).
2. Change the epochs (2 or 4) or the learning rate (1e-4).
3. Change the LoRA rank (r=16 or r=64), keeping alpha at 2×r.
4. Use only attention modules versus all modules.

Budget: at most 3 iterations (about 15 GPU hours) before reviewing the scope again.

Gate 6 (release): - ☐ Mean rubric score of 4.0 or higher, rule fidelity of 3.8 or higher, no criterion below 3.5. - ☐ Rule-fidelity quiz at 75% or higher; citation-ban violations at 2% or less. - ☐ Pairwise win rate of 70% or higher against the base model. - ☐ Safety at 100%. - ☐ No regression compared with the previous version.

Phase 7: Export and app

Goal: the fine-tuned model running on its own on the laptop, behind a small app.

#	Task	Output
7.1	Merge the adapter into the fp16 base (`merge_and_unload`)	`astro-merged-v1`
7.2	Convert to GGUF with llama.cpp; quantize to Q4_K_M and Q8_0	`.gguf` files
7.3	Write the Ollama `Modelfile` (Qwen template, system_v1, temperature 0.4, num_ctx 4096); run `ollama create astro-v1`	Local model
7.4	Check Q4_K_M quality against fp16 on 20 test items (expect less than a 0.2 drop in rubric score)	Check
7.5	Build `src/app/cli.py` : birth details → chart engine → `build_prompt` (uses `system_v1` and `render_chart_text`) → Ollama	App
7.6	End-to-end test: birth details → chart engine → answer, for 10 cases	Test log
7.7	Write `README.md` covering usage, limitations and a disclaimer	Docs

Gate 7: the full pipeline works on the laptop, speed is 5 or more tokens/second on CPU, and the disclaimer is shown to users.

---

2. Kaggle GPU budget

Use	Hours
Smoke test	about 1
Main run v1	about 2–3
Evaluation inference (base + adapter)	about 2–3
Iterations v2–v3	about 10–15
Total	**about 15–22 hours**, which fits in about 1 week of free quota

3. Roles

Role	Responsibility
Owner (you)	Decisions, human review, gate approvals
Claude Code	Scripts, notebooks, filters, eval tooling
Teacher LLM (Claude Opus 5.5)	Drafting questions and training answers
Judge LLM (Claude Opus 5.5)	Fact checks and rubric scoring for evaluation

4. Definition of done (v1)

1. `astro-v1` passes Gate 6 and Gate 7.
2. All artifacts (dataset hash, adapter, manifest, eval report) are in `runs/v1/`.
3. The project follows every principle in the Constitution, checked with the compliance checklist below.

Constitution compliance checklist

- ☐ I. Every training answer comes from a source passage; no chapter or verse citations or invented sources found in evaluation.
- ☐ II. All charts come from `pyswisseph`; the LLM does no calculation.
- ☐ III. Plain LoRA, fp16 base, trained on a Kaggle T4.
- ☐ IV. Human review done; the evaluation set was held out.
- ☐ V. Safety set at 100%; the disclaimer is shown.
- ☐ VI. The model stays private until the licence review is complete.
- ☐ VII. No real birth data in the dataset.
- ☐ VIII. Run manifest complete.
- ☐ IX. Beats the base model and the previous version.

5. Next actions (as of 2026-10-07)

1. **Spot-check (done 2026-10-07, short form):** the owner judged 5 examples (3 model-approved, 2 model-rejected) and agreed with the model review on all 5. Finding: one reject (#1) was a false reject. The reviewer called *Vedic Astrology: Rasi Characteristics* a fabricated title, but it is a corpus book. Fix: all 338 rejects re-reviewed with the corpus book list supplied (`src/dataset/rereview.py`). Outcome: the second look approved 199, but an audit showed most were the model changing its mind on genuine errors (e.g. a wrong Jupiter-Venus relationship), not title mistakes. Only example #1 was a pure title false-reject, so it was restored by a recorded manual override; the rest stay rejected. Final training data: **1,548** examples (1,421 train / 67 val / 60 test).
2. **Kaggle:** upload `kaggle/astro-sft/` as a private dataset named `astro-sft` , import `notebooks/train_lora.ipynb` , run the smoke test (`SMOKE_TEST = True`), then the full run (Phase 5).
3. **Terms check** for using Claude and Gemini outputs to train the model (task 0.7): still open.

6. Backlog (v2 and later)

Items deliberately deferred so they are not lost. Each needs a decision before it starts.

6.1 Hindi support (high priority for v2)

Why: many users' native language is Hindi, and Lal Kitab, one of the most popular texts among Hindi speakers, is currently excluded.

Books to recover (excluded from v1):

Book	Problem	Planned fix
`lal-kitab-1941`	Hindi text in a legacy font encoding (extracts as gibberish such as "fnu rd fQj ogh")	Legacy-font (Kruti Dev-style) to Unicode converter
`chamatkar-chintamani`	Hindi in a different legacy font encoding	Converter with its own character mapping
`lal-kitab-vol-1/2/3-1952`	Scanned Hindi pages	Tesseract Hindi pack (`hin.traineddata`), or a vision model (Claude/Gemini) for better quality on old scans

Tasks:

#	Task	Output
H.1	Feasibility test: convert a few pages of each legacy-font book and OCR a few pages of Lal Kitab 1952; the owner judges readability	Go/no-go per book
H.2	Decide the output form: Devanagari Hindi, Hinglish (Roman script), or both	Decision recorded
H.3	Amend the Constitution (v1.3): bilingual scope; update `system_v1` to "answer in the user's language" (as `system_v2`)	Constitution and prompt versions
H.4	Extract and chunk the recovered Hindi books (same pipeline, Hindi-aware quality checks)	Hindi chunks
H.5	Generate Hindi Q&A: (a) from the recovered Hindi books; (b) Hindi versions of questions on the existing 4,012 English passages	Hindi examples
H.6	Lal Kitab remedies (upayas): same "optional, never guaranteed" framing; reject extreme or harmful remedies	Teacher prompt rule
H.7	Review: model review plus a Hindi-speaking human spot-check by the owner	Review report
H.8	Train a bilingual adapter (v2) and compare it with v1 on English and Hindi test sets; note that Devanagari text uses more tokens per word	v2 adapter and eval report

Not to recover: `parashara-light-manual-6` (software manual) and `mathematics-of-astrology` (calculation is done by code, Constitution II).

6.2 Data quality follow-ups from round 1

1. **Tighten the answer prompt** for comparison and concept tasks: the model review rejected 36% of comparisons and 24% of concepts, mostly for improvised rules or misattributions. Also ban vague source references such as "this modern Vedic work" or "the book".
2. **Regenerate the 338 model-rejected examples** with the tightened prompt instead of discarding them (they are kept in `data/sft/filtered.jsonl` and `data/sft/auto_review.jsonl`).
3. **Grow the dataset** with the remaining ~2,200 planned seeds (round 2), using the same pipeline.
4. **Chart engine:** compare 10 charts against Jagannatha Hora by hand (open item from Gate 2).

6.3 Model and method options (from Tech Spec 3.4)

1. Continued pre-training (LoRA on raw text) before instruction tuning.
2. RAG, if exact citations or broader coverage become necessary.
3. A larger base model (8B or more) with QLoRA or paid GPUs.